Part A — Provided APK Analysis (10 marks)

1. **Task 1**: Unpack and Decompile the Network APK

• Use AI and your Assignment 1 workflow to decompile a2 case1.apk; confirm access to manifest, main activity, and at least one network-request class.

• Briefly document your decompilation steps and tools.

- Manifest: resources/AndroidManifest.xml
- Main activity: sources/com/example/mastg_test0019/MainActivity.java
- Network UI/request path: resources/res/layout/activity_main.xml

Jadx is used to decompile a2 case1.apk.

2. **Task 2**: Understand the App and Build a Network-Focused Model

• Ideally, run the app in an emulator/device and observe when/how network requests occur. If not possible, use tutor demos and static analysis.

• Summaries briefly what the app is trying to do and what information appears to travel over the network.

• Draw a simple system/threat model showing the app, the network, any servers it talks to, and an on-path attacker (e.g., someone on the same Wi-Fi).

App purpose:

- activity_main.xml contains a WebView.

- MainActivity loads remote web content, so the app behaves like a simple

network transport demo.

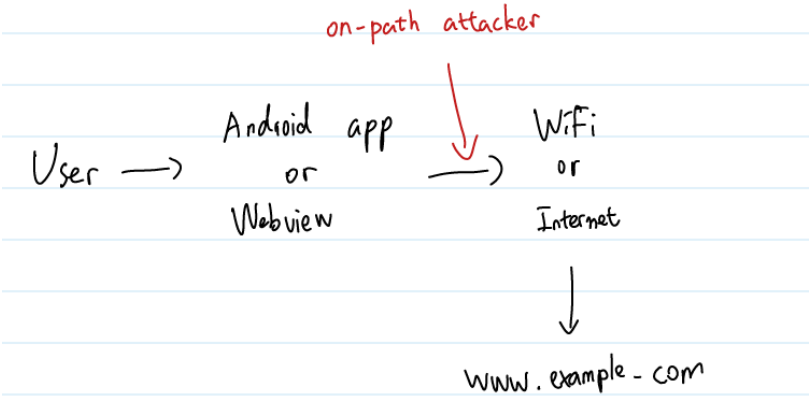
- Information that travels over the network:

  - The HTML page returned by the remote site.

  - Any linked resources fetched by the page.

  - Request metadata such as URL, headers, cookies, and any future credentials/tokens if the app is extended.

- Threat model:



3. **Task 3**: Find and Explain the Network Vulnerability

• Modified 11 Mar: Hint: the vulnerability we are primarily looking for, and the only vulnerability class that will be graded in Part A, is an insecure transport/TLS configuration issue that could enable MITM-style attacks.

• Use static analysis to identify insecure transport paths (e.g., cleartext traffic, disabled certificate/hostname checks).

• Show concise evidence (code snippets, file names, or config details) that

enables insecure network behavior.

• Explain, in plain language, what an on-path attacker can read/modify and why this is unsafe （including MITM relevance）.

• Describe realistic worst-case impact (e.g., data disclosure/modification, content injection, credential/session theft).

- Original vulnerable evidence captured from the provided decompiled code before remediation:

```java
// resources/AndroidManifest.xml
android:usesCleartextTraffic="true"

// sources/com/example/mastg_test0019/MainActivity.java
public void onReceivedSslError(...) {
    sslErrorHandler.proceed();
}

webView.loadUrl("http://www.example.com");

new HostnameVerifier() {
    public boolean verify(String str, SSLSession session) {
        return true;
    }
};
```

- Why this is vulnerable:
  - Cleartext HTTP allows anyone on the network path to read and modify traffic.
  - `sslErrorHandler.proceed()` accepts invalid certificates, defeating TLS certificate validation.
  - A `HostnameVerifier` that always returns `true` defeats hostname verification and accepts impostor servers.
- MITM impact:
  - Read the page content and request metadata.
  - Modify returned HTML or JavaScript and inject malicious content.
  - Phish credentials, steal cookies or session data, or mislead users with fake content.
  - If the app later handled secrets, both confidentiality and integrity would fail.
- Severity reasoning:
  - The app exposes a classic MITM path by disabling transport protections and server authentication.

4. **Task 4**: Mitigation Ideas

• Propose concrete mitigations (e.g., enforce encrypted/authenticated channels, tighten clear-text flags, enforce correct certificate/hostname validation).

• Briefly explain why the proposed changes reduce/remove risk for this app.

- Enforce HTTPS only.
- Set android:usesCleartextTraffic=false.
- Add a network security config with cleartextTrafficPermitted=false.
- Cancel SSL errors instead of proceeding.
- Remove custom hostname bypass logic and rely on platform validation.
- Optionally add certificate pinning for high-value production targets.